Chapter 17. Reflection, or "A Class Named Class"

17.0 Introduction

The class `java.lang.Class` and the reflection package java.lang.reflect provide a number of mechanisms for gathering information from the Java Virtual Machine. Known collectively as *reflection*, these facilities allow you to load classes on the fly, to find methods and fields in classes, to generate listings of them, and to invoke methods on dynamically loaded classes. There is even a mechanism to let you construct a class from scratch (well, actually, from an array of bytes) while your program is running. This is about as close as Java lets you get to the magic, secret internals of the Java machine.

The JVM itself is a large program, normally written in C and/or C++, that implements the Java Virtual Machine abstraction. You can get the source for OpenJDK and other JVMs via the internet, and you could study it for months. Here we concentrate on just a few aspects, and only from the point of view of a programmer using the JVM's facilities, not how it works internally; that is an implementation detail that could vary from one vendor's JVM to another.

I'll start with loading an existing class dynamically, move on to listing the fields and methods of a class and invoking methods, and end by creating a class on the fly using a `ClassLoader`. One of the more interesting aspects of Java, and one that accounts for its flexibility (applets in days of yore, servlets, web services, and other dynamic APIs) while also once being part of its perceived speed problem, is the concept of *dynamic loading*. For example, even the simplest "Hello, Java" program has to load the class file for your `HelloJava` class, the class file for its parent (usually `java.lang.Object`), the class for `PrintStream` (because you used `System.out`), the class for `PrintStream`'s parent, and `IOException`, and its parent, and so on. To see this in action, try something like this:

```
java -verbose HelloJava | more
```

To give another example, when applets were popular, a browser would download an applet's bytecode file over the internet and run it on your desktop. How does it load the class file into the running JVM? We discuss this little bit of Java magic in Recipe 17.1. The chapter ends with replacement versions of the JDK tools `javap` and a cross-reference tool that you can use to become a famous Java author by publishing your very own reference to the complete Java API.

17.1 Loading and Instantiating a Class Dynamically

Problem

You want to load classes dynamically, just like web servers load your servlets.

Solution

Use `class.forName("ClassName");` and the class's `newInstance( )` method.

Discussion

Suppose you are writing a Java application and want other developers to be able to extend your application by writing Java classes that run in the context of your application. In other words, these developers are, in essence, using Java as an extension language. You would probably want to define a small set of methods that these extension programs would have and that you could call for such purposes as initialization, operation, and termination. The best way to do this is, of course, to publish a given, possibly abstract, class that provides those methods and get the developers to subclass from it.

We'll leave the thornier issues of security and of loading a class file over a network socket for now and assume that the user can install the classes into the application directory or into a directory that appears in the `CLASSPATH` at the time the program is run. First, let's define our class. We'll call it `Cooklet` (see Example 17-1) to avoid infringing on the overused word *applet*. Pretend each subclass will represent the code to drive some elaborate kind of food-preparing-and-cooking appliance through the steps of one traditional recipe. And we'll initially take the easiest path from ingredients to cookies before we complicate it.

Example 17-1. Cooklet.java

```
/** A simple class, just to provide the list of methods that
 * users need to provide to be usable in our application.
 * Note that the class is abstract so you must subclass it,
 * but the methods are nonabstract so you don't have to provide
 * dummy versions if you don't need a particular functionality.
 */
public abstract class Cooklet {

    /** The initialization method. The Cookie application will
     * call you here (AFTER calling your no-argument constructor)
     * to allow you to initialize your code.
     */
    public void initialize( ) {
    }
```

```
    /** The work method. The cookie application will call you
     * here when it is time for you to start cooking.
     */
    public void work( ) {
    }

    /** The termination method. The cookie application will call you
     * here when it is time for you to stop cooking and shut down
     * in an orderly fashion.
     */
    public void terminate( ) {
    }
}
```

Now, because we'll be baking, er, making this available to other people, we'll probably want to cook up a demonstration version too; see Example 17-2.

Example 17-2. main/src/main/java/reflection/DemoCooklet.java

```
public class DemoCooklet extends Cooklet {
  public void work() {
    System.out.println("I am busy baking cookies.");
  }
  public void terminate() {
    System.out.println("I am shutting down my ovens now.");
  }
}
```

But how does our application use it? Once we have the name of the user's class, we need to create a `Class` object for that class. This can be done easily using the static method `Class.forName()`. Then we can create an instance of it using the `Class` object's `newInstance()` method; this calls the class's no-argument constructor. Then we simply cast the newly constructed object to our `Cooklet` class, and we can call its methods! It actually takes longer to describe this code than to look at the code, so let's do that now; see Example 17-3.

Example 17-3. main/src/main/java/reflection/Cookies.java

```
public class Cookies {
  public static void main(String[] argv) {
    System.out.println("Cookies Application Version 0.0");
    Cooklet cooklet = null;
    String cookletClassName = argv[0];
    try {
      @SuppressWarnings("unchecked")
      Class<Cooklet> cookletClass =
```

```
            (Class<Cooklet>) Class.forName(cookletClassName);
          cooklet = cookletClass.getConstructor().newInstance();
        } catch (Exception e) {
          System.err.println("Error " + cookletClassName + e);
        }
        cooklet.initialize();
        cooklet.work();
        cooklet.terminate();
      }
    }
```

And if we run it?

```
$ java Cookies DemoCooklet
Cookies Application Version 0.0
I am busy baking cookies.
I am shutting down my ovens now.
$
```

Of course, this version has rather limited error handling. But you already know how to fix that. Your `ClassLoader` can also place classes into a package by constructing a `Package` object; you should do this if loading any medium-sized set of application classes.

17.2 Printing Class Information

Problem

You want to print all the information about a class, similar to how it's done by `javap`.

Solution

Get a `Class` object, call its `getFields()` and `getMethods()`, and print the results.

Discussion

The JDK includes a program called `javap`, the Java printer. This normally prints the outline of a class file—a list of its methods and fields—but can also print out the Java bytecodes or machine instructions. The Kaffe package—a long gone reimplementation of the JDK—did not include a version of `javap`, so I wrote one and contributed it (see Example 17-4). The Kaffe folks expanded it somewhat, but at the end it still worked basically the same. My version didn't print the bytecodes; it behaves rather like the original version when you don't give it any command-line options.

The `getFields()` and `getMethods()` methods return arrays of `Field` and `Method`, respectively; these are both in the java.lang.reflect package. I use a `Modifiers` object to get details on the permissions and storage attributes of the fields and methods. In many Java implementations, you can bypass this and simply call `toString()` in each `Field` and `Method` object (as I do here for `Constructors`). Doing it this way gives me a bit more control over the formatting.

Example 17-4. main/src/main/java/reflection/MyJavaP.java

```
public class MyJavaP {

 /** Simple main program, construct self, process each class name
  * found in argv.
  */
 public static void main(String[] argv) {
  MyJavaP pp = new MyJavaP();

  if (argv.length == 0) {
   System.err.println("Usage: MyJavaP className [...]");
   System.exit(1);
  } else for (int i=0; i<argv.length; i++)
   pp.doClass(argv[i]);
 }

 /** Format the fields and methods of one class, given its name.
  */
 protected void doClass(String className) {
  try {
   Class<? extends Object> c = Class.forName(className);

   final Annotation[] annotations = c.getAnnotations();
   for (Annotation a : annotations) {
    System.out.println(a);
   }

   System.out.println(c + " {");

   Field fields[] = c.getDeclaredFields();
   for (Field f : fields) {
    final Annotation[] fldAnnotations = f.getAnnotations();
    for (Annotation a : fldAnnotations) {
     System.out.println(a);
    }
    if (!Modifier.isPrivate(f.getModifiers()))
     System.out.println("\t" + f + ";");
   }

   Constructor<? extends Object>[] constructors = c.getConstructors();
   for (Constructor<? extends Object> con : constructors) {
    System.out.println("\t" + con + ";");
```

```
      }

      Method methods[] = c.getDeclaredMethods();
      for (Method m : methods) {
        final Annotation[] methodAnnotations = m.getAnnotations();
        for (Annotation a : methodAnnotations) {
          System.out.println(a);
        }
        if (!Modifier.isPrivate(m.getModifiers())) {
          System.out.println("\t" + m + ";");
        }
      }
      System.out.println("}");
    } catch (ClassNotFoundException e) {
      System.err.println("Error: Class " +
        className + " not found!");
    } catch (Exception e) {
      System.err.println("JavaP Error: " + e);
    }
  }
}
```

17.3 Getting a Class Descriptor

Problem

You want to get a `Class` object from a class name or instance, perhaps to display this information or to use it to access the class dynamically.

Solution

Use the compiler keyword `.class` or the `getClass()` method, as appropriate.

Discussion

If the type name is known at compile time, you can get the class instance using the compiler keyword `.class`, which works on any type that is known at compile time, even the eight primitive types.

Otherwise, if you have an object (an instance of a class), you can call the `java.lang.Object` method `getClass()`, which returns the `Class` object for the object's class (now that was a mouthful!):

```
System.out.println("Trying the ClassName.class keyword:");
System.out.println("Object class: " + Object.class);
```

```
    System.out.println("String class: " + String.class);
    System.out.println("String[] class: " + String[].class);
    System.out.println("LocalDate class: " + LocalDate.class);
    System.out.println("Current class: " + ClassKeyword.class);
    System.out.println("Class for int: " + int.class);
    System.out.println();

    System.out.println("Trying the instance.getClass() method:");
    System.out.println("Sir Robin the Brave".getClass());
    System.out.println(LocalDate.now().getClass());
```

When we run it, we see this:

```
C:\javasrc\reflect>java ClassKeyword
java main/src/main/java/reflection/ClassKeyword.java
Trying the ClassName.class keyword:
Object class: class java.lang.Object
String class: class java.lang.String
String[] class: class [Ljava.lang.String;
LocalDate class: class java.time.LocalDate
Current class: class reflection.ClassKeyword
Class for int: int

Trying the instance.getClass() method:
class java.lang.String
class java.time.LocalDate

C:\javasrc\reflect>
```

Nothing fancy, but as you can see, you can get the `Class` object for almost any type that is known at compile time, whether it's part of a package or not.

17.4 Finding and Using Methods and Fields

Problem

You need to find arbitrary method or field names in arbitrary classes.

Solution

Use the reflection package java.lang.reflect.

Discussion

If you just wanted to find fields and methods in one particular class, you wouldn't need this recipe; you could simply create an instance of the class using `new` and refer to its fields and methods directly. But the Reflection API allows you to find methods and fields in any class, even classes that have not yet been written! Given a class object created as in Recipe 17.3, you can obtain a list of constructors, a list of methods, or a list of fields. The method `getMethods()` lists the methods available for a given class as an array of `Method` objects. Similarly, `getFields()` returns a list of `Field` objects. Because constructor methods are treated specially by Java, there is also a `getConstructors()` method, which returns an array of `Constructor` objects. Even though `Class` is in the package java.lang, the `Constructor`, `Method`, and `Field` objects it returns are in java.lang.reflect, so you need an import of this package. The `ListMethods` class (see Example 17-5) shows how to get a list of methods in a class when only its name is known at runtime.

Example 17-5. main/src/main/java/reflection/ListMethods.java

```
public class ListMethods {
 public static void main(String[] argv) throws ClassNotFoundException {
  if (argv.length == 0) {
   System.err.println("Usage: ListMethods className");
   return;
  }
  Class<?> c = Class.forName(argv[0]);
  Constructor<?>[] cons = c.getConstructors();
  printList("Constructors", cons);
  Method[] meths = c.getMethods();
  printList("Methods", meths);
 }
 static void printList(String s, Object[] o) {
  System.out.println("*** " + s + " ***");
  for (int i=0; i<o.length; i++)
   System.out.println(o[i].toString());
 }
}
```

For example, you could run Example 17-5 on a class like `java.lang.String` and get a fairly lengthy list of methods; I'll only show part of the output so you can see what it looks like:

```
> java reflection.ListMethods java.lang.String
*** Constructors ***
public java.lang.String( )
public java.lang.String(java.lang.String)
public java.lang.String(java.lang.StringBuffer)
public java.lang.String(byte[])
// and many more...
*** Methods ***
```

```
public static java.lang.String java.lang.String.copyValueOf(char[])
public static java.lang.String java.lang.String.copyValueOf(char[],int,int)
public static java.lang.String java.lang.String.valueOf(char)
// and more valueOf( ) forms...
public boolean java.lang.String.equals(java.lang.Object)
public final native java.lang.Class java.lang.Object.getClass( )
// and more java.lang.Object methods...
public char java.lang.String.charAt(int)
public int java.lang.String.compareTo(java.lang.Object)
public int java.lang.String.compareTo(java.lang.String)
```

You can see that this could be extended (almost literally) to write a `BeanMethods` class that would list only the set/get methods defined in a bean class.

Alternatively, you can find a particular method and invoke it, or find a particular field and refer to its value. Let's start by finding a given field, because that's the easiest. Example 17-6 is code that, given an `Object` and the name of a field, finds the field (gets a `Field` object) and then retrieves and prints the value of that `Field` as an `int`.

Example 17-6. main/src/main/java/reflection/FindField.java

```
public class FindField {

  public static void main(String[] unused)
  throws NoSuchFieldException, IllegalAccessException {

    // Create instance of FindField
    FindField gf = new FindField();

    // Create instance of target class (YearHolder defined below).
    Object o = new YearHolder();

    // Use gf to extract a field from o.
    System.out.println("The value of 'currentYear' is: " +
      gf.intFieldValue(o, "currentYear"));
  }

  int intFieldValue(Object o, String name)
  throws NoSuchFieldException, IllegalAccessException {
    Class<?> c = o.getClass();
    Field fld = c.getField(name);
    int value = fld.getInt(o);
    return value;
  }
}

/** This is just a class that we want to get a field from */
class YearHolder {
```

```
  /** Just a field that is used to show getting a field's value. */
  public int currentYear = LocalDate.now().getYear();
}
```

What if we need to find a method? The simplest way is to use the methods `getMethod()` and `invoke()`. But this is not altogether trivial. Suppose that somebody gives us a reference to an object. We don't know its class but have been told that it should have this method:

```
public void work(String s) { }
```

We wish to invoke `work()`. To find the method, we must make an array of `Class` objects, one per item in the parameter list. So, in this case, we make an array containing only a reference to the class object for `String`. Because we know the name of the class at compile time, we'll use the shorter invocation `String.class` instead of `Class.forName()`. This, plus the name of the method as a string, gets us entry into the `getMethod()` method of the `Class` object.

If this succeeds, we have a `Method` object. But guess what? In order to invoke the method, we have to construct yet another array, this time an array of `Object` references actually containing the data to be passed to the invocation. We also, of course, need an instance of the class in whose context the method is to be run. For this demonstration class, we need to pass only a single string, because our array consists only of the string. Example 17-7 is the code that finds the method and invokes it.

Example 17-7. main/src/main/java/reflection/GetAndInvokeMethod.java

```
/**
 * Get a given method, and invoke it.
 */
public class GetAndInvokeMethod {

  /** This class is just here to give us something to work on,
   * with a println() call that will prove we got into it.
   */
  static class X {
    public void work(int i, String s) {
      System.out.printf("Called: i=%d, s=%s%n", i, s);
    }
    // The main code
    public static void main(String[] args) {
      System.out.println("Main.args = " + Arrays.toString(args));
    }
  }

  public static void main(String[] argv) {
    try {
```

```
      Class<?> clX = X.class; // or Class.forName("X");

      // To find a method we need the array of matching Class types.
      Class<?>[] argTypes = {
        int.class,
        String.class
      };

      // Now find a Method object for the given method.
      Method worker = clX.getMethod("work", argTypes);

      // To INVOKE the method, we need the invocation
      // arguments as an Object array.
      Object[] theData = {
        42,
        "Chocolate Chips"
      };

      // The obvious last step: invoke the method.
      // First arg is an instance, null if static method
      worker.invoke(new X(), theData);

      System.out.println("First Invoke Done");

      // Same deal but for main, a static method taking String[]:
      final Method m = clX.getMethod("main", String[].class);
      final Object[] args = new Object[1];
      args[0] = "Hello World Of Java".split(" ");
      // e.g., args[0] is itself an array
      m.invoke(null, args);
      System.out.println("Second Invoke Done");

    } catch (Exception e) {
      System.err.println("Invoke() failed: " + e);
      e.printStackTrace();
    }
  }
}
```

Not tiny, but it's still not bad. In most programming languages, you couldn't do that in the 40 lines it took us here.

WARNING

When the arguments to a method are of a primitive type, such as `int`, you do not pass `Integer.class` into `getMethod()`. Instead, you must use the class object representing the primitive type `int`. The easiest way to find this class is in the `Integer` class, as a public constant named `TYPE`, so you'd pass `Integer.TYPE`. The same is true for all the primitive types; for each, the corresponding wrapper `class` has the primitive `class` referred to as `TYPE`.

Another caution: it's an advanced topic, but there are some interactions between the module system and the Reflection API related to permissions and intermodule reflection.

Java also includes a mechanism called a `MethodHandle` that was intended to both simplify and generalize the use of Reflection to invoke methods; we cover it in the following recipe, Recipe 17.5.

17.5 Invoking Class Members via MethodHandles

Problem

You hope to find an easy way to find and invoke methods, with the possibility of transforming arguments being passed into them.

Solution

Consider using the MethodHandle API. It is intended to replace the Reflection API for invoking methods, is a bit easier to use, and is generally faster.

Discussion

As with many such APIs, there is a lookup class that must be created initially. You then create a `MethodType` describing the signature (one return type and any number of argument types) of the method you are focused on. These are then used to look up a method and create a `MethodHandle` for it. You then call the methods using the `invoke()` or `invokeExact()` method on the `MethodHandle`, passing required arguments.

A simple example (after imports from `java.lang.invoke`) creates a `MethodType` for a method returning a string given two `int`s. On the next line we see that this is going to be used to invoke the `substring()` method, and finally we see the invocation, to extract the substring establishment from the longer string provided, as shown in Example 17-8.

Example 17-8. main/src/main/java/reflection/MethodHandleAbcs.java

```
MethodHandles.Lookup lookup = MethodHandles.lookup();
MethodType mt = MethodType.methodType(String.class, int.class, int.class);
MethodHandle mh = lookup.findVirtual(String.class, "substring", mt);
String s = (String) mh.invokeExact("Antidisestablishmentarianism", 7, 20);
System.out.println(s);  // prints "establishment"
```

Example 17-9 contains a more complete example.

Example 17-9. main/src/main/java/reflection/MethodHandleDemo.java

```
// Create a Lookup object
MethodHandles.Lookup lup = MethodHandles.lookup();

// First invoke LocalDate's static "of(year, month, day)" method:

// Create a matcher for the arg list types we want to invoke
// First the return type, then the argument types
MethodType mt = MethodType.methodType(LocalDate.class,
   int.class, int.class, int.class);
// Use the Lookup object to find the static method "of" in LocalDate
MethodHandle mh = lup.findStatic(LocalDate.class, "of", mt);
// At last, invoke the method!
LocalDate endDate = (LocalDate) mh.invokeExact(2014, 6, 10);
System.out.println("LocalDate from 'of' = " + endDate);

// Second, invoke the LocalDate's instance method toString()
// "toString()" returns a String, takes no args
mt = MethodType.methodType(String.class);
// Find the instance method in the LocalDate class
mh = lup.findVirtual(LocalDate.class, "toString", mt);
// Invoke it in the context of the LocalDate object created earlier
String asString = (String) mh.invokeExact(endDate);
System.out.println("LocalDate as String is " + asString);
```

As always, the Javadoc for the `MethodHandle` class contains more detail.

17.6 Listing Classes in a Package

Problem

You want to get a list of all the classes in a package.

Solution

You can't, in the general sense. There are some limited approaches, most involving `CLASSPATH` scanning.

Discussion

There is no way to find out all the classes in a package, in part because, as we just saw in Recipe 17.9, you can add classes to a package at any time! And, for better or worse, the JVM and standard classes such as `java.lang.Package` do not even allow you to enumerate the classes currently in a given package.

The nearest you can come is to look through the `CLASSPATH`. And this will surely work only for local directories and JAR files; if you have locally defined or network-loaded classes, this is not going to help. In other words, it will find compiled classes, but not dynamically loaded classes. There are several libraries that can automate this for you, and you're welcome to use them. The code to scan the `CLASSPATH` is fairly simple at heart, though, so classy developers with heart will want to examine it. Example 17-10 shows my `ClassesInPackage` class with its one static method. The code works but is rather short on error handling, and it will crash on nonexistent packages and other failures.

The code goes through a few gyrations to get the `CLASSPATH` as an enumeration of URLs, then looks at each element.

file

URLs will contain the pathname of the file containing the *.class* file, so we can just list it.

jar

URLs contain the filename as *file:/path_to_jar_file!package/name*, so we have to pull this apart; the *package/name* suffix is slightly redundant in this case because it's the package we asked the `ClassLoader` to give us.

Example 17-10. main/src/main/java/reflection/ClassesInPackage.java

```
public class ClassesInPackage {

 /** This approach began as a contribution by Paul Kuit at
  * http://stackoverflow.com/questions/1456930/, but his only
  * handled single files in a directory in classpath, not in Jar files.
  * N.B. Does NOT handle system classes!
  * @param packageName
  * @return
  * @throws IOException
  */
```

```
public static String[] getPackageContent(String packageName)
 throws IOException {

 final String packageAsDirName = packageName.replace(".", "/");
 final List<String> list = new ArrayList<>();
 final Enumeration<URL> urls =
    Thread.currentThread().
    getContextClassLoader().
    getResources(packageAsDirName);
 while (urls.hasMoreElements()) {
  URL url = urls.nextElement();
  // System.out.println("URL = " + url);
  String file = url.getFile();
  switch (url.getProtocol()) {
  case "file":
    // This is the easy case: "file" is
    // the full path to the classpath directory
    File dir = new File(file);
    for (File f : dir.listFiles()) {
     list.add(packageAsDirName + "/" + f.getName());
    }
    break;
  case "jar":
    // This is the harder case; "file" is of the form
    // "jar:/home/ian/bleah/darwinsys.jar!com/darwinsys/io"
    // for some jar file that contains at least one class from
    // the given package.
    int colon = file.indexOf(':');
    int bang = file.indexOf('!');
    String jarFileName = file.substring(colon + 1, bang);
    JarFile jarFile = new JarFile(jarFileName);
    Enumeration<JarEntry> entries = jarFile.entries();
    while (entries.hasMoreElements()) {
     JarEntry e = entries.nextElement();
     String jarEntryName = e.getName();
     if (!jarEntryName.endsWith("/") &&
      jarEntryName.startsWith(packageAsDirName)) {
      list.add(jarEntryName);
     }
    }
    break;
  default:
    throw new IllegalStateException(
    "Dunno what to do with URL " + url);
  }
 }
 return list.toArray(new String[] {});
}

public static void main(String[] args) throws IOException {
```

```
    String[] names = getPackageContent("com.darwinsys.io");
    for (String name : names) {
      System.out.println(name);
    }
    System.out.println("Done");
  }
}
```

Note that if you run this application in the *javasrc* project, it will list the members of the demonstration package (com.darwinsys.io) twice, because it will find them both in the build directory and in the JAR file. If this is an issue, change the `List` to a `Set` (see Recipe 7.4)—the reason that different data structures exist!

17.7 Accessing Nested Members of Same Class

Problem

You want to access methods declared in your outer class or in another of its nested classes.

Solution

Access private members in containing classes just as you did before Java 11. Use the nest-based API to get information on nest mates.

Discussion

Java has long supported nested classes, also called inner classes. A problem arose when an inner class needed to access a private member of the outer class. Since this was forbidden by the JVM's access rules, the compiler had to insert a hidden "bridge" method in the outer class, to be called by the inner class. Example 17-11 shows an example of *nest mates*, a term that encompasses the host (outer) class and any nested (inner) classes.

Example 17-11. main/src/main/java/lang/NestMates.java

```
/**
 * Illustrate "nest mates" access control. Compile this on
 * both Java 1.8 and Java 11, and compare the .class files.
 */
public class NestMates {

  private void privateProcess() {
    System.out.println("Hello from a private method.");
  }
```

```
  public class Nested {
    public void innerProcess() {
      privateProcess();
    }
  }

  Nested nested = new Nested();

  public static void main(String[] args) {
    new NestMates().nested.innerProcess();
  }
}
```

The inner class invokes a private method in the outer class. Prior to Java 11, this was technically prohibited by the JVM specification. However, the Java compiler would insert a hidden "bridge" method in the outer class and generate a call to it whenever the inner class tried to call the outer private method. As of Java 11, the JVM rules were altered so that this access became legal—nest mates are allowed to touch each other's private members—and the bridge method is no longer needed.

The bottom line is that you don't have to change anything in this class when moving from Java 8 to Java 11—the compiler simply handles access differently.

You can view the change in the generated JVM bytecode by running Java 8 and a Java 11 or later `javac` command, and viewing the compiled code with `javap -c`. An example of this is in the shell script `nestmatesdemo` in the same directory as Example 17-11.

Since the "nest-mated-ness" may be of interest to some tools, the Reflection API was extended to allow explicit access to nest mates. The `Class` class added the following methods:

- `Class getNestHost()` returns the host of the nest.
- `boolean isNestMateOf(Class c)` returns `true` if `this` and `c` are in the same nest.
- `Class[] getNestMembers()` returns an array of `Class` for all members of the nest.

These are briefly demonstrated in Example 17-12.

Example 17-12. main/src/main/java/lang/NestMatesReflection.java

```
void main() {
  Class outer = lang.NestMates.class;
  Class inner = lang.NestMates.Nested.class;
  System.out.println("Is outer a nestmate of itself? " + outer.isNestmateOf(outer));
  System.out.println("Is outer a nestmate of inner? " + outer.isNestmateOf(inner));
  System.out.println("What is inner's nest host? " + inner.getNestHost());
  System.out.println("What are all of inner's nestmates? " +
```

```
    java.util.Arrays.toString(inner.getNestMembers()));
}
```

17.8 Accessing Private Methods and Fields via Reflection

Problem

You want to access private fields and have heard you can do so using the Reflection API.

Solution

It's generally a bad idea to access private fields. If you have to, you can. This mechanism is really intended for tool builders who make IDEs, debuggers, and certain libraries.

Discussion

There is occasionally a need to access private fields in other classes. For example, I did so recently while writing a JUnit test case that needed to see all the fields of a target class. The secret is to call the `Field` or `Method` descriptor's `setAccessible()` method, passing the value `true` before trying to get the value or invoke the method. It really is that easy, as shown in Example 17-13.

Example 17-13. main/src/main/java/reflection/DefeatPrivacy.java

```
class ClassWithPrivateField {
  @SuppressWarnings("unused") // Used surreptitiously below.
  private int p = 42;
  int q = 3;
}

/**
 * Demonstrate that it is, in fact, all too easy to access private members
 * of an object using Reflection.
 */
public class DefeatPrivacy {

  public static void main(String[] args) throws Exception {
    new DefeatPrivacy().process();
  }

  private void process() throws Exception {
    ClassWithPrivateField x = new ClassWithPrivateField();
    System.out.println(x);
    // System.out.println(x.p); // Won't compile
```

```
    System.out.println(x.q);
    Class<? extends ClassWithPrivateField> class1 = x.getClass();
    Field[] flds = class1.getDeclaredFields();
    for (Field f : flds) {
     f.setAccessible(true);  // bye-bye "private"
     System.out.println(f + "==" + f.get(x));
     f.setAccessible(false);  // reset to "correct" state
    }
   }
  }
```

WARNING

Use this with *extreme care*, because it can defeat some of the most cherished principles of Java programming.

17.9 Constructing a Class from Scratch with a ClassLoader

Problem

You need to load a class from a nonstandard location and run its methods.

Solution

Examine the existing loaders such as `java.net.URLClassLoader`. If none are suitable, write and use your own `ClassLoader`.

Discussion

A `ClassLoader`, of course, is a class that loads classes. A few `ClassLoader`s are built into the Java Virtual Machine, but your application can create others as needed. Learning to write and run a working `ClassLoader` and using it to load a class and run its methods is a nontrivial exercise. In fact, you rarely need to write a `ClassLoader`, but knowing how is helpful in understanding how the JVM finds classes, creates objects, and calls methods.

A given class loader will have a *namespace* associated with it. For example, in a Java web server, there can easily be multiple web applications, and several of them might have a class with the same full name. In a single basic application this would be a conflict, but with multiple `ClassLoader`s, one per web app, there is no conflict, as each has its own namespace.

`ClassLoader` itself is abstract; you must subclass it, presumably providing a `loadClass()` method that loads classes as you wish. It can load the bytes from a network connection, a local disk, RAM, a serial port, or anywhere else. Or you can construct the class file in memory yourself, if you have access to a compiler.

There is a general-purpose loader called `java.net.URLClassLoader` that can be used if all you need is to load classes via the web protocol (or, more generally, from one or more URLs).

You must call the `ClassLoader` `loadClass()` method for any classes you wish to explicitly load from it. Note that this method is called to load all classes required for classes you load (superclasses that aren't already loaded, for example). However, the JVM still loads classes that you instantiate with the `new` operator normally via `CLASSPATH`.

When writing a `ClassLoader`, your `loadClass()` method needs to get the class file into a byte array (typically by reading it), convert the array into a `Class` object, and return the result.

What? That sounds a bit like "and then a miracle occurs..." And it is. The miracle of class creation, however, happens down inside the JVM, where you don't have access to it. Instead, your `ClassLoader` has to call the `protected defineClass()` method in your superclass (which is `java.lang.ClassLoader`). This is illustrated in Figure 17-1, where a stream of bytes containing a hypothetical `Chicken` class is converted into a ready-to-run `Chicken` class in the JVM by calling the `defineClass()` method.

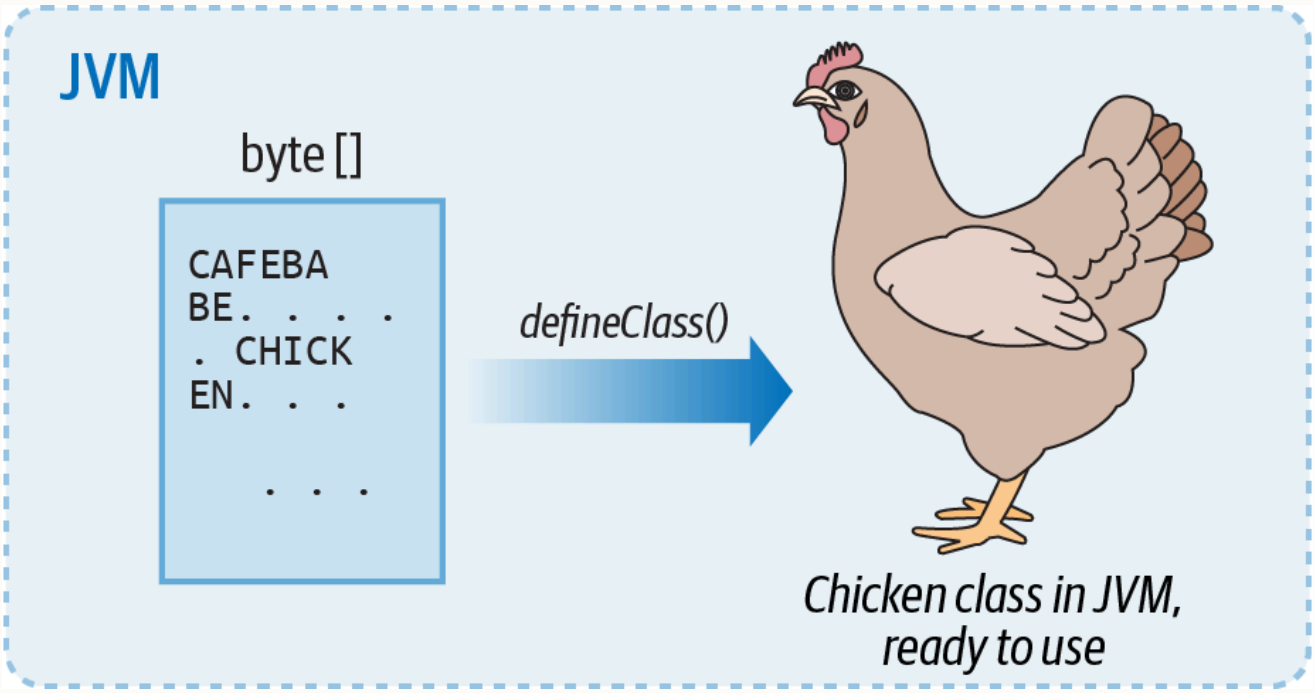


Figure 17-1. `ClassLoader` in action

What next?

To use your `ClassLoader` subclass, you need to instantiate it and call its `loadClass()` method with the name of the class you want to load. This gives you a `Class` object for the named class; the

`Class` object in turn lets you construct instances, find and call methods, etc. Refer back to Recipe 17.4.

17.10 Constructing a Class from Scratch with JavaCompiler

Problem

You'd rather construct a class dynamically by generating and compiling source code.

Solution

Use the `JavaCompiler` from javax.tools.

Discussion

There are many situations in which you might need to generate code on the fly. If you're writing a framework, you might want to introspect on a model class to find its fields and generate accessors for them on the fly. As we've seen in Recipe 17.4, you can do this with the `Field` class. However, for a high-volume operation it may be more efficient to generate direct access code.

The Java Compiler API has been around since Java 1.6 and is fairly easy to use for simple cases. Here are the basic steps:

- Get the `JavaCompiler` object for your current Java runtime. If it's not available, either give up altogether or fall back to using reflection.
- Get a `CompilerTask` (which is also a `Callable`) to run the compilation, passing input and outputs.
- Invoke the `Callable`, either directly or by using an `ExecutorService`.
- Check the results. If true, invoke the class.

This is demonstrated in Example 17-14.

Example 17-14. main/src/main/java/reflection/JavaCompilerDemo.java

```
import javax.tools.JavaCompiler;
import javax.tools.SimpleJavaFileObject;
import javax.tools.ToolProvider;

/** Demo the Java Compiler API: Create a class, compile, load, and run it.
 * Best run standalone using "java JavaCompiler.java"
 */
public class JavaCompilerDemo {
```

```
  private final static String PACKAGE = "reflection";
  private final static String CLASS = "AnotherDemo";
  private static boolean verbose;
  public static void main(String[] args) throws Exception {
    String source = "package " + PACKAGE + ";\n" +                    ❶
      "public class " + CLASS + " {\n" +
      "\tpublic static void main(String[] args) {\n" +
      "\t\tString message = (args.length > 0 ? args[0] : \"Hi\")" + ";\n" +
      "\t\tSystem.out.println(message + \" from AnotherDemo\");\n" +
      "\t}\n}\n";
    if (verbose)
      System.out.print("Source to be compiled:\n" + source);

    JavaCompiler compiler = ToolProvider.getSystemJavaCompiler(); ❷
    if (compiler == null) {
      throw new IllegalStateException("No default compiler, giving up.");
    }
    Callable<Boolean> compilation =
      compiler.getTask(null, null, null, List.of("-d","."), null, ❸
      List.of(new MySource(CLASS, source)));
    boolean result = compilation.call();                    ❹
    if (result) {
      System.out.println("Compiled OK");
      Class<?> c = Class.forName(PACKAGE + "." + CLASS);          ❺
      System.out.println("Class = " + c);
      Method m = c.getMethod("main", args.getClass());            ❻
      System.out.println("Method descriptor = " + m);
      Object[] passedArgs = { args };
      m.invoke(null, passedArgs);                           ❼
    } else {
      System.out.println("Compilation failed");
    }
  }
}
```

❶ The source code that we want to compile. In real life it would probably be dynamically generated, maybe using a `StringBuilder`.

❷ Get a reference to the default `JavaCompiler` object.

❸ Ask the compiler to create a `CompilerTask` to do the compilation. `CompilerTask` is also `Callable` and we save it under that type. The `-d` and `.` are standard `javac` arguments. `MySource` extends the compiler-provided API class `SimpleJavaFileObject` to give access to a file by creating a *file://* URL.

❹ A `Callable` can be put into a thread pool (`ExecutorService`) (see Recipe 11.1); we don't need this capability but the Compiler API returns it. We invoke the `Callable` directly.

❺ Assuming the `result` was `true` indicating success, we load the class with `Class.forName()`.

❻ We have to find the `main()` method in the generated class. We reuse the `String[].class` type from `args`, since all `main` methods have the same argument.

❼ Finally, we can invoke the `main` method, reusing the incoming `args` array to pass any *welcome* message along.

Running this program with and without an argument shows that the argument passed to the `JavaCompilerDemo` is being passed correctly to the generated `AnotherDemo` class:

```
$ java src/main/java/reflection/JavaCompilerDemo.java
Compiled OK
Class = class reflection.AnotherDemo
Method descriptor = public static void
  reflection.AnotherDemo.main(java.lang.String[])
Hi from AnotherDemo
$ java src/main/java/reflection/JavaCompilerDemo.java Welcome
Compiled OK
Class = class reflection.AnotherDemo
Method descriptor = public static void
  reflection.AnotherDemo.main(java.lang.String[])
Welcome from AnotherDemo
$
```

There is a lot to explore in the Compiler API, including the `JavaFileManager` that lets you control the placement of class files (other than by using `-d` as we did here), listeners to monitor compilation, and control of output and error streams. Consult the `javax.tools.JavaCompiler` documentation for details.

17.11 Constructing or Modifying Class Files with the Class-File API 22P

Problem

You want to examine, create, or modify *.class* class files, perhaps in a library, in the tooling of an IDE, in a dynamic interpretation environment, etc.

Solution

Use the new Class-File API, defined in the java.lang.classfile package.

Discussion

The Reflection API has stood us in good stead since before Java 1.0, but there are limits on what it can do. It is primarily used to inspect (or "introspect") and load existing classes. To modify classes, we've had to use third-party libraries such as ASM, a self-proclaimed "all purpose Java bytecode manipulation and analysis framework," which does allow you to create and/or modify Java bytecode. In fact, a copy of ASM has traditionally been bundled as part of the JDK. However, the Java team thought they could do better, and the result is the java.lang.classfile package. Built using modern idioms such as lambdas, this API allows you to:

- Inspect existing class files (when referred to as an array of bytes, not in-memory like the Reflection API)
- Create new class files out of whole cloth—this is intended to be used by compilers, including `javac`
- Modify existing class files at the structural level
- Modify class files at the bytecode or "assembly language" level

For example, aspect-oriented programming (AOP) requires the construction of wrapper classes dynamically at runtime. The new Class-File API is ideal for this. Example 17-15 shows the use of this API to inspect a class file.

Example 17-15. main/src/main/java/classfileapi/ClassFileAPIDemo.java

```
@MyAnnotation
class ClassFileAPIDemo implements Serializable {
 int number = 0;
 void main() throws IOException {
  byte[] classFile = Files.readAllBytes(
   Path.of("target/classes/classfileapi/ClassFileAPIDemo.class"));
  ClassModel model = ClassFile.of().parse(classFile);
  for (ClassElement element : model) {
   switch (element) {
    case FieldModel field ->
     System.out.println(
     "Field " + field.fieldName().stringValue());
    case MethodModel method ->
     System.out.println(
     "Method " + method.methodName().stringValue());
    default -> {
     /* Ignore anything else */
     System.out.println(
     "Other: " + element +
      "(" + element.getClass().getName() + ")");
    }
   }
  }
```

```
 }
}
```

The output is fairly voluminous:

```
$ java --enable-preview -cp target/classes classfileapi.ClassFileAPIDemo
Other: AccessFlags[flags=32] (jdk.internal.classfile.impl.AccessFlagsImpl)
Other: ClassFileVersion[majorVersion=66, minorVersion=65535]
      (jdk.internal.classfile.impl.ClassFileVersionImpl)
Other: Superclass[superclassEntry=java/lang/Object]
      (jdk.internal.classfile.impl.SuperclassImpl)
Other: Interfaces[interfaces=java/io/Serializable] (jdk.internal.classfile.impl.InterfacesImpl)
Field number
Method <init>
Method main
Other: Attribute[name=SourceFile]
      (jdk.internal.classfile.impl.BoundAttribute$BoundSourceFileAttribute)
Other: Attribute[name=RuntimeVisibleAnnotations]
      (jdk.internal.classfile.impl.BoundAttribute$BoundRuntimeVisibleAnnotationsAttribute)
Other: Attribute[name=InnerClasses] (jdk.internal.classfile.impl.BoundAttribute$BoundInnerClassesAttribute)
$
```

This class examines its own *.class* file, just so we don't have to keep a dummy file around. You'll notice not only the expected `Method` and `Field` listings, but also many other bits of low-level information, such as the class file version, the source file, the annotation applied to the main class (that's the `RuntimeVisibleAnnotations`), and the definition of the annotation (the last line).

As a longer example, Example 17-16 uses this API to *create a class file entirely within the code*, saves it to disk as a new *.class* file for analysis, and runs its main method.

Example 17-16. main/src/main/java/classfileapi/CreateLoadAndRun.java

```
import static java.lang.constant.ConstantDescs.*;

/**
 * Create a canonical "Hello, World" class using the Class-File API;
 * load it with a ClassLoader, and invoke main() via Reflection.
 * @author Class-File part based on example in Javadoc
 * for java.lang.classfiles
 * @author Glue around that by Ian Darwin
 */
public class CreateLoadAndRun extends ClassLoader {

  final String fullClassName = "notapackage.Hello";  ❶
  // A few ClassDescs that are not in ConstantDescs  ❷
  final ClassDesc CD_fullClassName = ClassDesc.of(fullClassName);
  final ClassDesc CD_System = ClassDesc.of(System.class.getName());
```

```
  final ClassDesc CD_PrintStream =
    ClassDesc.of(PrintStream.class.getName());
  // A few MethodDescs that are not in ConstantDescs
  final MethodTypeDesc MTD_void_String =
    MethodTypeDesc.of(CD_void, CD_String);
  final MethodTypeDesc MTD_void_StringArray =
    MethodTypeDesc.of(CD_void, CD_String.arrayType());

  void main() throws Exception {
    byte[] classData = ClassFile
      .of()
      .build(CD_fullClassName,                    ❸
      clb -> clb.withFlags(ClassFile.ACC_PUBLIC)

        // requires no-arg constructor to be valid   ❹
        .withMethod(ConstantDescs.INIT_NAME, ConstantDescs.MTD_void,
          ClassFile.ACC_PUBLIC,
          mb -> mb.withCode(
            cob -> cob.aload(0)                   ❺
              .invokespecial(ConstantDescs.CD_Object,
                ConstantDescs.INIT_NAME, ConstantDescs.MTD_void)
              .return_()))

        // Standard "void main(String[])" main method
        .withMethod("main", MTD_void_StringArray,    ❻
          ClassFile.ACC_PUBLIC + ClassFile.ACC_STATIC,
          methodBuilder -> methodBuilder.withCode(   ❼
            cob -> cob.getstatic(CD_System, "out", CD_PrintStream)
              .ldc("Hello World")
              .invokevirtual(CD_PrintStream, "println", MTD_void_String)
              .return_())));

    // Save the new class to disk, just so we can examine & verify
    var dirPath = Path.of("/tmp/notapackage");       ❽
    if (!Files.exists(dirPath)) {
      Files.createDirectory(dirPath);
    }
    Files.write(Path.of("/tmp/notapackage/Hello.class"), classData);

    // Define the class in current ClassLoader
    // (e.g., this) and then call main()
    Class<?> c =                                     ❾
      defineClass(fullClassName, classData, 0, classData.length);

    Method m = c.getMethod("main", String[].class);

    m.invoke(null, new Object[]{new String[0]});
  }
}
```

❶ Full class name includes package. Package notapackage is just for humor.

❷ The creators of this tech use the non-Java-standard but reasonable convention of giving `ClassDesc` objects names beginning `CD_` and `MethodTypeDesc`s names with `MTD_`. Most of the ones we need are statically imported from `ConstantDesc`s.

❸ Start of the call to `build()`; we pass the `CD` for the class name, and a class builder object `clb`, which creates the class as `public`.

❹ Start construcing the no-arg constructor. Every class must have at least one constructor, and a "main program" class typically doesn't need constructors with arguments. `INIT_NAME` is the special name `<init>` used for constructors.

❺ Here we use a code builder `cob` to generate actual machine code ("bytecode") instructions: load a zero value, call the superclass constructor, and return.

❻ This begins the usual `public static void main(String[])` method.

❼ This code builder finds the static field `System.out` of type `PrintStream`, loads the String "Hello World" onto the stack as a parameter, invokes the `PrintStream` method `println()`, and returns.

❽ Straightforward I/O operations. See Chapter 10.

❾ Standard Reflection API. See Recipe 17.4.

This is a very general API and has the ability to transform class files, for example, when a package name changes (as in the Java EE classes all being renamed from "javax." to "jakarta." when Oracle transferred Java EE to the Eclipse Software Foundation). It can do many other types of transformation, such as inserting method calls, removing them, adding or removing individual bytecode files, and more. The reader is referred to the Javadoc for this class; the package-level file contains more details and examples of many transforms.

17.12 Using and Defining Annotations

Problem

You need to know how to use annotations in code or how to define your own annotations.

Solution

Apply annotations in your code using `@AnnotationName` before a class, method, field, etc. Define annotations with `@interface` at the same level as `class`, `interface`, etc.

Discussion

Annotations are a way of adding additional information beyond what the source code conveys. Annotations may be directed at the compiler or at runtime examination. Their syntax was somewhat patterned after Javadoc annotations (such as `@author`, `@version` inside doc comments). Annotations are what I call *class-like things* (so they have initial-cap names) but are prefixed by the `@` sign where used (e.g., `@Override`). You can place them on classes, methods, fields, and a few other places; they must appear immediately before what they annotate (ignoring space and comments). A given annotation may appear only once in a given position (this is relaxed in Java 8 or 9).

As an example of the benefits of a compile-time annotation, consider the common error made when overriding: as shown in Example 17-17, a small error in the method signature can result it an overload when an override was intended.

Example 17-17. MyClass.java (an example of why we need annotations)

```
public class MyClass {

   public boolean equals(MyClass object2) {
      // compare, return boolean
   }
}
```

The code will compile just fine on any release of Java, but it is *incorrect*. The standard contract of the `equals()` method (see Recipe 8.1) requires a method whose solitary argument is of type `java.lang.Object`. The preceding version creates an accidental overload. Because the main use of `equals()` (and its buddy method `hashCode()`; see Recipe 8.1) is in the `Collections` classes (see Chapter 7), this overloaded method will never get called, resulting in both dead code and incorrect operation of your class within `Set`s and `Map`s.

The solution is very simple: using the annotation `java.lang.Override`, as in Example 17-18, informs the compiler that the annotated method is required to override a method inherited from a supertype (such as a superclass or an interface). If not, the code will not compile.

Example 17-18. MyClass.java with `@Override` annotation

```
public class MyClass {

   @Override
   public boolean equals(MyClass object2) {
      // compare, return boolean
   }
}
```

This version of `equals()`, while still incorrect, will be flagged as erroneous at compile time, potentially avoiding a lot of debugging time. This annotation, on your own classes, will help both at the time you write new code and as you maintain your codebase; if a method is removed from a superclass, all the subclasses that still attempt to override it *and* have the `@Override` annotation will cause an error message, allowing you to remove a bunch of dead code.

The second major use of annotations is to provide metadata at runtime. For example, the Jackson JSON APIs in Recipe 16.2 provide configuration information to its `ObjectMapper`. The Jakarta Persistence API (JPA, see information on my website) uses its own set of annotations from the package javax.persistence to mark up entity classes to be loaded and/or persisted. The Spring Framework lets developers use annotations to provide a range of information

I said that annotations are class-like things, and therefore, you can define your own. The syntax here is a bit funky; you use `@interface`. It is rumored that the team developing this feature was either told not to, or was afraid to, introduce a new keyword into the language, due to the trouble that doing so had caused when the `enum` keyword was introduced in Java SE 1.4. Or maybe they just wanted to use a syntax that was more reminiscent of the annotation's usage. At any rate, Example 17-19 is a trivial example of a custom annotation.

Example 17-19. Trivial annotation defined

```
package lang;

public @interface MyToyAnnotation {
}
```

Annotations are class-like things, so they should be named the same way—that is, names that begin with a capital letter, and, if public, they are stored in a source file of the same name (e.g, *MyToyAnnotation.java*).

Compile the code in Example 17-19 with `javac` and you'll see there's a new *MyToyAnnotation.class* file. In Example 17-20, we examine this with `javap`, the standard JDK class inspection tool.

Example 17-20. Running `javap` on trivial annotation

```
$ javap lang.MyToyAnnotation
Compiled from "MyToyAnnotation.java"
public interface lang.MyToyAnnotation extends java.lang.annotation.Annotation {
}
$
```

As it says, an `Annotation` is represented in the class file format as just an interface that extends `Annotation` (to answer the obvious question, you could write simple interfaces this way, but it

would be a truly terrible idea). In Example 17-21, we take a quick look at `Annotation` itself.

Example 17-21. The `Annotation` interface in detail

```
$ javap java.lang.annotation.Annotation
Compiled from "Annotation.java"
public interface java.lang.annotation.Annotation {
  public abstract boolean equals(java.lang.Object);
  public abstract int hashCode();
  public abstract java.lang.String toString();
  public abstract java.lang.Class<? extends java.lang.annotation.Annotation>
    annotationType();
}
$
```

Annotations can be made such that the compiler will only allow them in certain points in your code. Example 17-22 is an annotation that can only go on classes or interfaces.

Example 17-22. Sample annotation for classes, interfaces, etc.

```
@Target(ElementType.TYPE)
@Retention(RetentionPolicy.RUNTIME)
public @interface MyAnnotation {
}
```

The `@Target` specifies where the annotation can be used: `ElementType.TYPE` makes it usable on classes, interfaces, class-like things such as enums, even annotations! To restrict it to use on just annotations, there is `ElementType.ANNOTATION_TYPE`. Other types include `METHOD`, `FIELD`, `CONSTRUCTOR`, `LOCAL_VARIABLE`, `PACKAGE`, and `PARAMETER`. So, this annotation is itself annotated with two `@ANNOTATION_TYPE`-targeted annotations.

Usage of annotations with an existing framework requires consulting its documentation. Using annotations for your own purpose at runtime requires use of the Reflection API, as shown in Example 17-23.

One more thing to note about annotations is that they may have attributes. These are defined as methods in the annotation source code but used as attributes where the annotation is used. Example 17-23 is an annotated annotation with one such attribute.

Example 17-23. main/src/main/java/lang/AnnotationDemo.java

```
/**
 * A sample annotation for types (classes, interfaces);
 * it will be available at run time.
 */
```

```
@Target(ElementType.TYPE)
@Retention(RetentionPolicy.RUNTIME)
public @interface AnnotationDemo {
  public boolean fancy() default false;
  public int order() default 42;
}

/** A simple example of using the annotation */
@AnnotationDemo(fancy=true)
@Resource(name="Dumbledore")
class FancyClassJustToShowAnnotation {

  /** Print out the annotations attached to this class */
  public static void main(String[] args) {
    Class<?> c = FancyClassJustToShowAnnotation.class;
    System.out.println("Class " + c.getName() + " has these annotations:");
    for (Annotation a : c.getAnnotations()) {
      if (a instanceof AnnotationDemo ad) {
        System.out.println("\t" +a +
          " with fancy=" + ad.fancy() +
          " and order " + ad.order());
      } else {
        System.out.println("\tSomebody else's annotation: " + a);
      }
    }
  }
}
```

`AnnotationDemo` has the meta-annotation `@Target(ElementType.TYPE)` to indicate that it can annotate user-defined types (such as classes). Other `ElementType` choices include `METHOD`, `FIELD`, and `PARAMETER`. If more than one is needed, use array initializer syntax.

`AnnotationDemo` also has the `@Retention(RetentionPolicy.RUNTIME)` annotation to request that it be preserved until runtime. This is obviously required for any annotation that will be examined by a framework at runtime.

These two meta-annotations are common on user-defined annotations that will be examined at runtime.

The class `FancyClassJustToShowAnnotation` shows the use of `AnnotationDemo` annotation, along with a standard Java one (the `@Resource` annotation). Refer to Recipe 17.13 for a full example of using this mechanism.

17.13 Finding Plug-In-Like Classes via Annotations

Problem

You want to load plug-in classes without using an explicit plug-in API.

Solution

Define an annotation for the purpose, and use it to mark the plug-in classes.

Discussion

Suppose we want to model how the Java EE standard `javax.annotations.Named` or `javax.faces.ManagedBean` annotations work. For each class that is so annotated, convert the class name to an instance-like name (e.g., lowercase the first letter), and do something special with it. You'd want to do something like the following:

1. Get the list of classes in the given package(s) (see Recipe 17.6).
2. Check if the class is annotated.
3. If so, save the name and `Class` descriptor for later use.

This is implemented in Example 17-24.

Example 17-24. main/src/main/java/reflection/PluginsViaAnnotations

```
/** Discover "plug-ins" or other add-in classes via Reflection using Annotations */
public class PluginsViaAnnotations {

 /**
  * Find all classes in the given package which have the given
  * class-level annotation class.
  */
 public static List<Class<?>> findAnnotatedClasses(String packageName,
  Class<? extends Annotation> annotationClass) throws Exception {

  List<Class<?>> ret = new ArrayList<>();
  String[] clazzNames = ClassesInPackage.getPackageContent(packageName);
  for (String clazzName : clazzNames) {
   if (!clazzName.endsWith(".class")) {
    continue;
   }
   clazzName = clazzName.replace('/', '.').replace(".class", "");
   Class<?> c = null;
   try {
    c = Class.forName(clazzName);
```

```
    } catch (ClassNotFoundException ex) {
      System.err.println("Weird: class " + clazzName +
        " reported in package but gave CNFE: " + ex);
      continue;
    }
    if (c.isAnnotationPresent(annotationClass) &&
        !ret.contains(c))
        ret.add(c);

   }
   return ret;
  }
```

We can take this one step further and support particular method annotations, similar to `javax.annotations.PostCreate`, which is meant to decorate a method that is to be called after an instance of the bean has been instantiated by the framework. Our flow is now something like this, and the code is shown in Example 17-25:

1. Get the list of classes in the given package(s) (again, see Recipe 17.6).
2. If you are using a class-level annotation, check if the class is annotated.
3. If this class is still of interest, get a list of its methods.
4. For each method, see if it contains a given method-specific annotation.
5. If so, add the class and method to a list of invocable methods.

Example 17-25. main/src/main/java/reflection/PluginsViaAnnotations (find annotated methods)

```
/**
 * Find all classes in the given package which have the given
 * method-level annotation class on at least one method.
 */
public static List<Class<?>> findClassesWithAnnotatedMethods(String packageName,
    Class<? extends Annotation> methodAnnotationClass) throws Exception {
 List<Class<?>> ret = new ArrayList<>();
 String[] clazzNames = ClassesInPackage.getPackageContent(packageName);
 for (String clazzName : clazzNames) {
   if (!clazzName.endsWith(".class")) {
     continue;
   }
   clazzName = clazzName.replace('/', '.').replace(".class", "");
   Class<?> c = null;
   try {
     c = Class.forName(clazzName);
     // System.out.println("Loaded " + c);
   } catch (ClassNotFoundException ex) {
     System.err.println("Weird: class " + clazzName +
       " reported in package but gave CNFE: " + ex);
     continue;
```

```
    }
    for (Method m : c.getDeclaredMethods()) {
      // System.out.printf("Class %s Method: %s\n",
      //     c.getSimpleName(), m.getName());
      if (m.isAnnotationPresent(methodAnnotationClass) &&
          !ret.contains(c)) {
        ret.add(c);
      }
    }
  }
  return ret;
}
```

See Also

Recipe 17.12 and the rest of this chapter.

17.14 A Timing Program

Problem

You want to let Java report on the actual time it takes to run a given class.

Solution

Use `System.currentTimeMillis()` before and after invoking the class via the Reflection API.

Discussion

It's pretty easy to build a simplified `time` command in Java, given that you have `System.currentTimeMillis()` to start with. Run my `Time` program, and, on the command line, specify the name of the class to be timed, followed by the arguments (if any) that class needs for running. The program is shown in Example 17-26. The time that the class took is displayed. But remember that `System.currentTimeMillis()` returns clock time, not necessarily CPU time. So you must run it on a machine that isn't running a lot of background processes. Note also that I use dynamic loading (see Recipe 17.1) to let you put the Java class name on the command line.

Example 17-26. main/src/main/java/performance/Time.java

```
public class Time {
  public static void main(String[] argv) throws Exception {
    // Instantiate target class, from argv[0]
    Class<?> c = Class.forName(argv[0]);
```

```
    // Find its static main method (use our own argv as the signature).
    Class<?>[] classes = { argv.getClass() };
    Method main = c.getMethod("main", classes);

    // Make new argv array, dropping class name from front.
    // (Normally Java doesn't get the class name, but in
    // this case the user puts the name of the class to time
    // as well as all its arguments...)
    String nargv[] = new String[argv.length - 1];
    System.arraycopy(argv, 1, nargv, 0, nargv.length);

    Object[] nargs = { nargv };

    System.err.println("Starting class " + c);

    // About to start timing run. Important to not do anything
    // (even a println) that would be attributed to the program
    // being timed, from here until we've gotten ending time.

    // Get current (i.e., starting) time
    long t0 = System.currentTimeMillis();

    // Run the main program
    main.invoke(null, nargs);

    // Get ending time, and compute usage
    long t1 = System.currentTimeMillis();

    long runTime = t1 - t0;

    System.err.println(
      "runTime="  + Double.toString(runTime/1000D));
  }
}
```

Of course, you can't directly compare the results from the operating system `time` command with results from running this program. There is a rather large, but fairly constant, initialization overhead—the JVM startup and the initialization of `Object` and `System.out`, for example—that is included in the former and excluded from the latter. One could even argue that my `Time` program is more accurate because it excludes this constant overhead. But, as noted, it must be run on a single-user machine to yield repeatable results. And no fair watching cat videos in another window while waiting for your timed program to complete!

See Also

Optimizing Cloud Native Java by Benjamin Evans and James Gough (O'Reilly) explains how to tune Java cloud applications for performance "using a quantitative, verifiable, and repeatable ap-

proach," addressing various topics that can lead to optimal performance of Java applications running in the cloud. The older *Java Performance* by Charlie Hunt and Binu John (O'Reilly) provides information on tuning Java performance.

17.15 Program: CrossRef

You've probably seen those other Java books that consist entirely of listings of the Java API for version thus-and-such of the JDK. I don't suppose you thought the authors of these works sat down and typed the entire contents from scratch. As a programmer, you would have realized, I hope, that there must be a way to obtain that information from Java. But you might not have realized how easy it is! If you've read this chapter faithfully, you now know that there is one true way: make the computer do the walking. `CrossRef` is a program that puts most of these techniques together. This version generates a cross-reference listing, but by overriding the last few methods, you can easily convert it to print the information in any format you like, including an API reference book. You'd need to deal with the details of this or that publishing software—FrameMaker, troff, TEX, etc.—but that's the easy part.

This program makes fuller use of the Reflection API than did `MyJavaP` in Recipe 17.2. It also uses the `java.util.zip` classes (see Recipe 10.15) to crack the JAR archive containing the class files of the API. Each class file found in the archive is loaded and listed; the listing part is similar to `MyJavaP` (shown in Recipe 17.2).

`CrossRef` is also designed as a base class for subclassing. The methods `startClass()` and `endClass()` are empty. These methods are placeholders designed to make subclassing easy when you need to write something at the start and end of each class. One example is a fancy text-formatting application in which you need to output a bold header at the beginning of each class. Another might be XML, where you want to write `<class>` at the front of each class and `</class>` at the end. `CrossRefXML` is an XML-specific subclass that generates (limited) XML for each field and method.

Both are included in the repository, in the *main/src/main/java/reflection* directory. Both are also in the javasrc GitHub repository, in the *https://github.com/IanDarwin/javasrc/blob/main/main/src/main/java/reflection* directory. By the way, if you publish a book using either of these and get rich, "Remember, remember me!"

See Also

We have not investigated all the ins and outs of reflection or the `ClassLoader` mechanism, but by now you should have a basic idea of how it works. Perhaps the most important omissions are `SecurityManager` and `ProtectionDomain`. The `SecurityManager` was for decades the guardian of access to dangerous APIs. However, it is now deprecated for removal, as in, "terminate with extreme prejudice," and will be removed in a near-future JDK, at the same time as the ancient

`Applet` mechanism. A `ProtectionDomain` can be provided with a `ClassLoader` to specify all the permissions needed for the class to run.

I've also left unexplored many topics in the JVM; see the (somewhat dated) O'Reilly books *Java Virtual Machine* by Troy Downing and Jon Meyer, and *Java Language Reference* by Mark Grand. You can also read the Sun/Oracle *Java Language Specification* and *JVM Specification* documents (both updated with new releases, available online), for a lifetime of reading enjoyment and edification!

The Apache Software Foundation maintains a vast array of useful software packages that are free to access and use. The source code is available without charge from the Apache website. Two packages you might want to investigate include the Commons BeanUtils and the Byte Code Engineering Library (BCEL). Commons BeanUtils claims to provide easier-to-use wrappers around some of the Reflection API. BCEL is a third-party toolkit for building and manipulating bytecode class files. Written by Markus Dahm, BCEL has become part of the Apache Commons Project.

Previous chapter
< 16. Processing JSON Data

Next chapter
18. Using Java with Other Languages >